

# 문자열 · JSON · 예외처리

# LLM 개발 핵심 문법

Python 기반 LLM 애플리케이션 개발을 위한 필수 문법

문자열 조작, f-string 프롬프트 생성, JSON 파싱, 안전한 API 호출 처리까지  
실무에서 바로 활용 가능한 핵심 기술을 학습합니다

# 목차

## 01 문자열 기초 및 메서드

- 문자열 생성 및 조작
- 주요 메서드 활용
- API 응답 정리 흐름

## 02 f-string 동적 프롬프트

- f-string 기본 문법
- 포맷 지정 방법
- 프롬프트 동적 생성

## 03 멀티라인 프롬프트 템플릿

- 삼중 따옴표 활용
- Few-shot 템플릿
- 이중 중괄호 사용법

## 04 JSON 파싱 및 변환

- json.loads/dumps
- 파싱 실패 패턴
- safe\_parse\_json 함수

## 05 예외처리 및 재시도 패턴

- try/except 구조
- 지수 백오프
- safe\_api\_call 함수

## 06 실습 및 강사 유의사항

- 체크포인트 확인
- 강사 시연 포인트
- 다음 단계 연결

# 학습 목표

## 문자열 조작

- 문자열 연결, 슬라이싱
- strip, replace 메서드
- API 응답 정리 패턴

## 동적 프롬프트

- f-string 변수 삽입
- 포맷 지정 활용
- 도메인별 프롬프트 생성

## 템플릿 작성

- 멀티라인 f-string
- Few-shot 프롬프트
- 이중 중괄호 JSON 출력

## JSON 변환

- json.loads 딕셔너리 변환
- json.dumps 문자열 변환
- ensure\_ascii 한글 처리

## 안전 처리

- try/except 에러 처리
- 지수 백오프 재시도
- API 호출 안전 패턴



# 문자열 기초



## 문자열 만들기

- 작은따옴표(')와 큰따옴표(") 모두 사용 가능
- 결과는 동일하며 상황에 따라 선택



## 따옴표 포함하기

- 문자열 안에 따옴표를 포함할 때는 바깥과 안쪽의 따옴표 종류를 다르게 사용
- 예: '그는 "안녕"이라고 말했다'



## LLM 코드에서의 중요성

- LLM 코드에서 다루는 데이터의 대부분은 문자열
  - 프롬프트, 응답, 질문, 답변 모두 문자열로 처리
-

# 자주 쓰는 문자열 메서드

`.strip()`

- 앞뒤 공백·줄바꿈 제거
- API 응답 정리에 필수
- 예: ' 텍스트 ' → '텍스트'

`.replace()`

- 특정 문자열을 다른 문자열로 치환
- JSON 코드펜스 제거
- 예: '` ``json' → "

`.split()`

- 구분자 기준으로 문자열 분리
- 리스트로 반환
- 예: 'a,b,c' → ['a','b','c']

`.lower()`

- 모든 문자를 소문자로 변환
- 대소문자 구분 없는 비교
- 예: 'Hello' → 'hello'

`.startswith()`

- 특정 문자로 시작하는지 확인
- True/False 반환
- 코드펜스 감지에 활용

# API 응답 정리 흐름

## 1단계: 앞뒤 공백 제거

- .strip() 메서드로 응답 문자열의 앞뒤 공백과 줄바꿈 제거
- API가 반환한 불필요한 여백 정리
- 깔끔한 데이터 전처리의 시작

## 2단계: 코드펜스 제거

- .replace('` ` `json', '') 로 마크다운 코드펜스 시작 부분 제거
- .replace('` ` `', '') 로 코드펜스 종료 부분 제거
- JSON 파싱 준비 완료

## 3단계: 마무리 정리

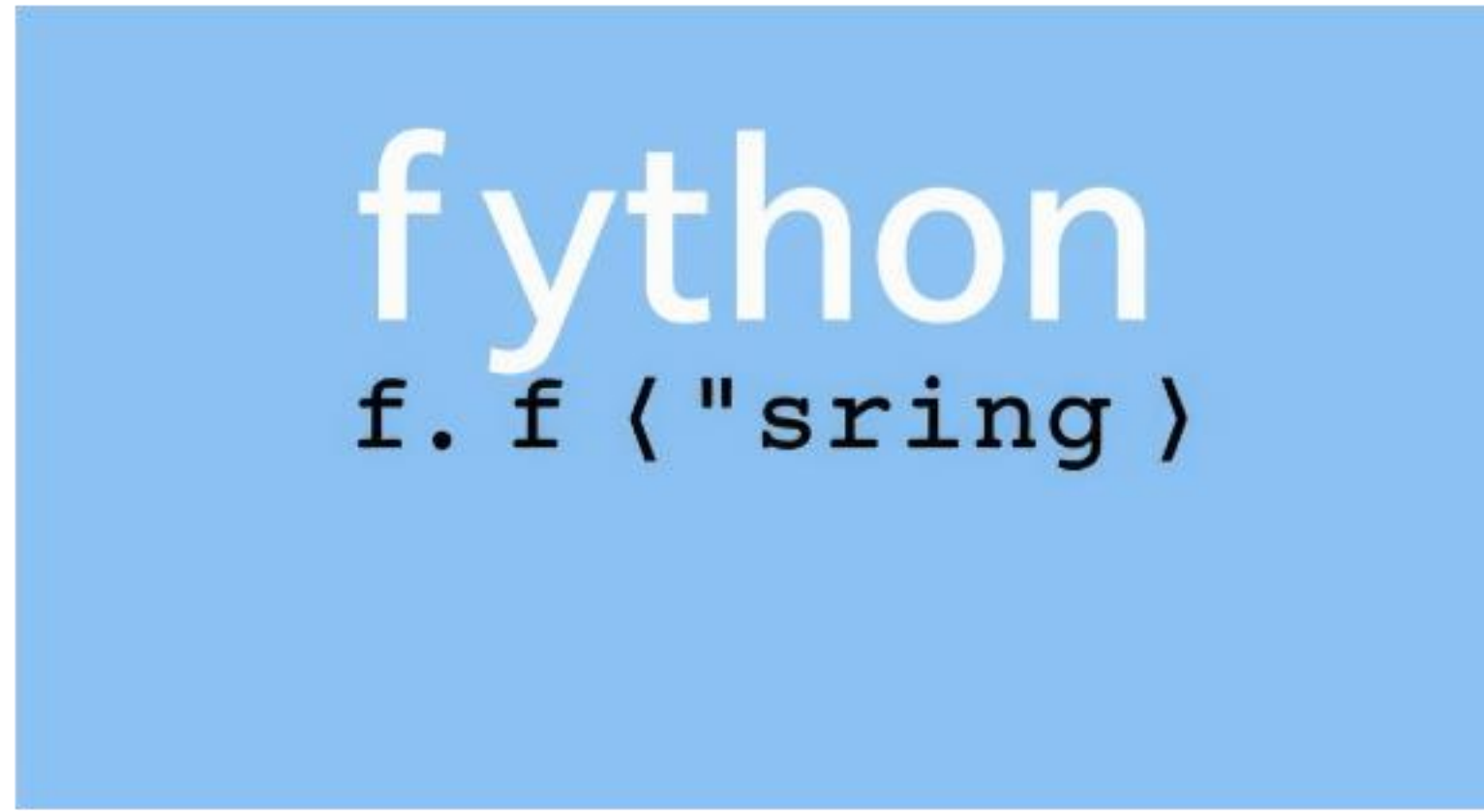
- 코드펜스 제거 후 다시 .strip() 실행
- 남아있을 수 있는 공백 최종 정리
- 파싱 가능한 순수 JSON 문자열 확보

## 4단계: Day 2 활용

- Day 2 Judge 평가 자동화에서 매번 사용하는 패턴
- safe\_parse\_json 함수로 구현
- Day 2~5 전체 과정에서 재사용



# f-string이란?



## f-string의 정의

- f-string은 문자열 앞에 f를 붙이고 중괄호 {} 안에 변수나 표현식을 넣어 동적 문자열을 생성하는 방법입니다
- 예: f'안녕 {name}'
- 현재 파이썬에서 권장되는 문자열 포매팅 방식입니다
- 가독성이 높아 코드 작성과 유지보수가 쉽습니다

## 문자열 생성 방식 비교

- + 연결: 복잡해지면 읽기 어려움 (비권장)
- % 방식: 구식 방법으로 현재는 사용하지 않음 (비권장)
- .format(): 가능하나 장황함
- f-string: 현재 표준이자 가장 권장되는 방식
- LLM 프롬프트 작성 시 필수적으로 사용됩니다

## 중괄호 활용법

- 중괄호 {} 안에 변수명을 넣으면 해당 변수의 값이 삽입됩니다
- 변수뿐 아니라 계산식, 함수 호출, 메서드도 사용 가능합니다
- 예시: f'{score/total\*100:.1f}%'
- 동적 프롬프트 생성의 핵심 기능입니다

# f-string 고급 활용

## 표현식과 함수 활용

- 중괄호 안에 계산식 직접 사용 가능: `f'{score/total*100}%'`
- 함수 호출 가능: `f'{len(text)}글자'`
- 메서드 사용 가능: `f'{name.upper()}'`
- 조건식도 삽입 가능: `f'{status if valid else 오류}'`

## 포맷 지정 - 소수점과 천 단위

- `:.1f` → 소수점 1자리: `f'{3.14159:.1f}' = '3.1'`
- `:.2f` → 소수점 2자리: `f'{3.14159:.2f}' = '3.14'`
- `:,` → 천 단위 쉼표: `f'{1234567:,}' = '1,234,567'`
- Day 1 API 실습에서 토큰 사용량 출력 시 활용됩니다

## 정렬 포맷

- `:<10` → 왼쪽 정렬 10칸
- `:>10` → 오른쪽 정렬 10칸
- `:^10` → 가운데 정렬 10칸
- 공백을 채워 깔끔한 출력 형식을 만들 수 있습니다

## 실무 활용 예시

- 토큰 사용량: `f'사용 토큰: {tokens:,}개'`
- 비용 계산: `f'비용: ${cost:.2f}'`
- 진행률: `f'진행률: {done/total*100:.1f}%'`
- Day 1 오후 API 실습부터 바로 사용합니다





# 멀티라인 f-string

## 삼중 따옴표와 f-string 조합

- 삼중 따옴표('''')로 여러 줄 문자열 작성이 가능합니다
- f-string과 함께 사용하여 긴 프롬프트를 구조화합니다
- 예시: `prompt = f'''당신은 {role}입니다...'''`
- 실무에서 복잡한 프롬프트 작성 시 필수입니다

## .strip()의 필요성

- 삼중 따옴표 시작 시 첫 줄에 불필요한 줄바꿈이 생깁니다
- 마지막 줄에도 여분의 줄바꿈이 포함됩니다
- .strip()으로 앞뒤 공백과 줄바꿈을 제거해야 합니다
- 깔끔한 프롬프트 완성을 위해 반드시 필요합니다

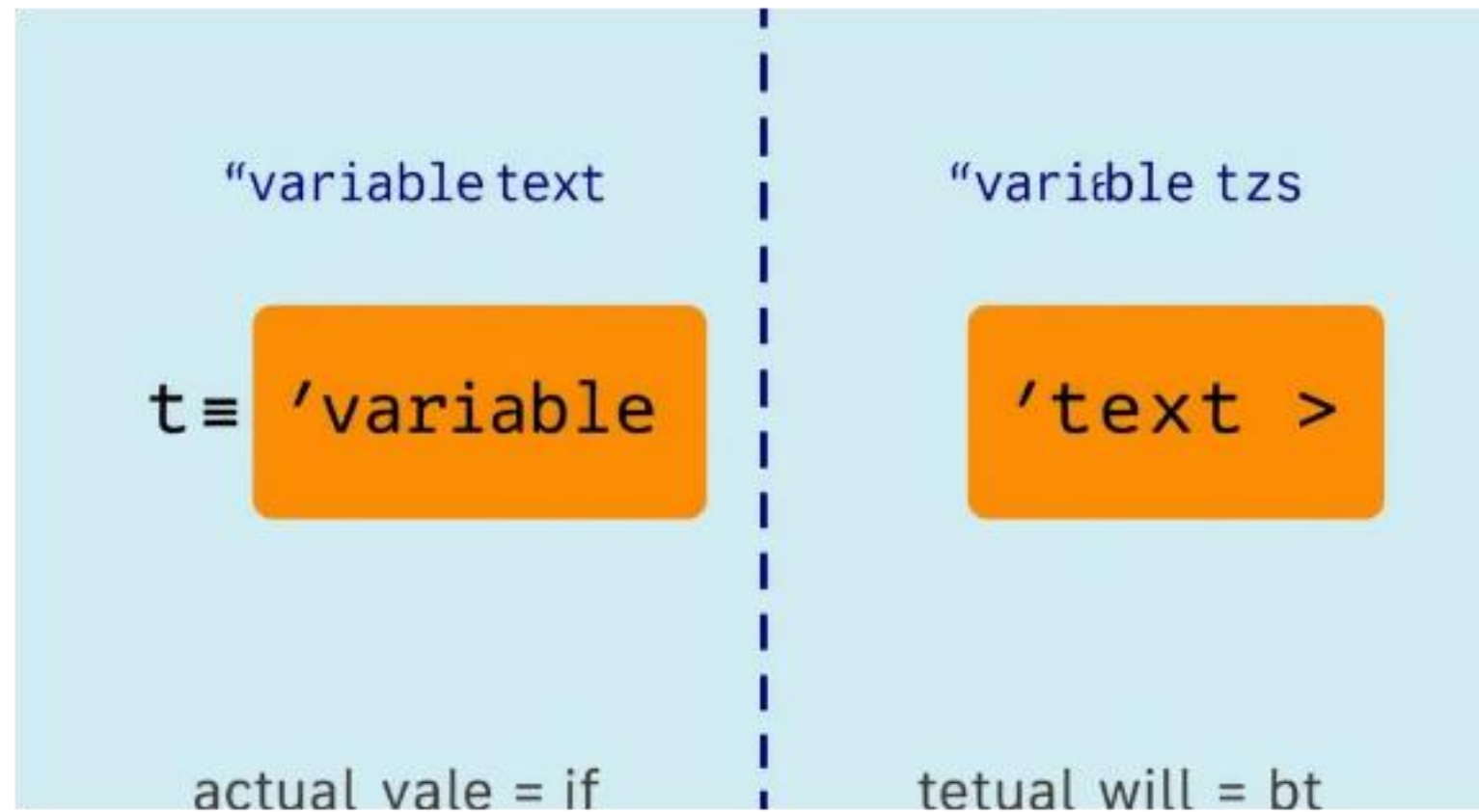
## 가독성 높은 프롬프트 작성

- 각 섹션을 명확히 구분하여 작성합니다
- 역할, 지시사항, 예시, 출력 형식을 단계별로 배치합니다
- 들여쓰기와 줄바꿈으로 구조를 시각화합니다
- 유지보수와 수정이 용이한 프롬프트를 만들 수 있습니다

## Few-shot 프롬프트 구성

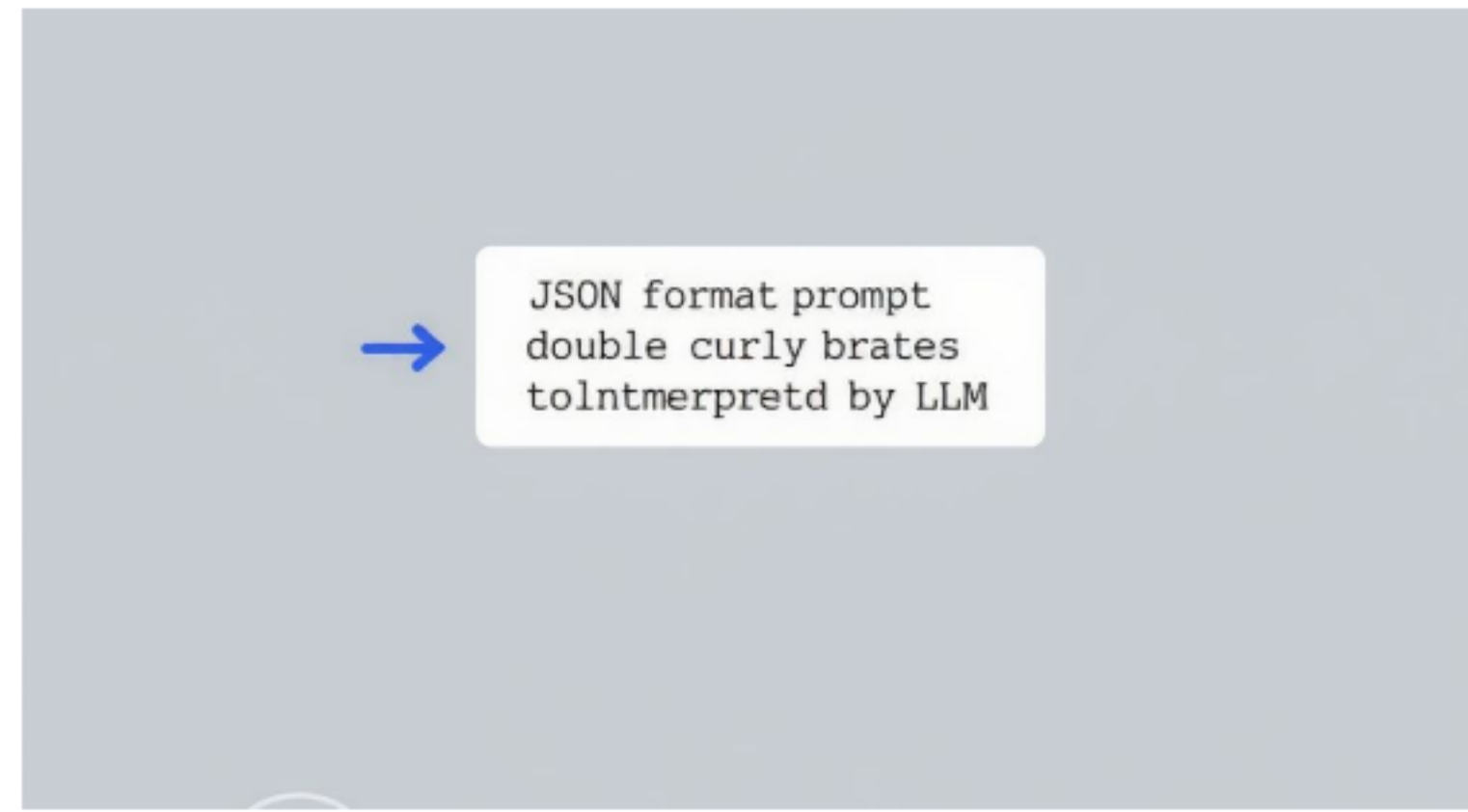
- Few-shot 학습: 예시를 여러 개 제공하는 프롬프트 기법입니다
- 멀티라인 f-string으로 예시들을 구조화합니다
- 각 예시에 변수를 삽입하여 동적으로 생성합니다
- Day 2 합성 데이터 생성에서 활용됩니다

# 이중 중괄호 {{}}



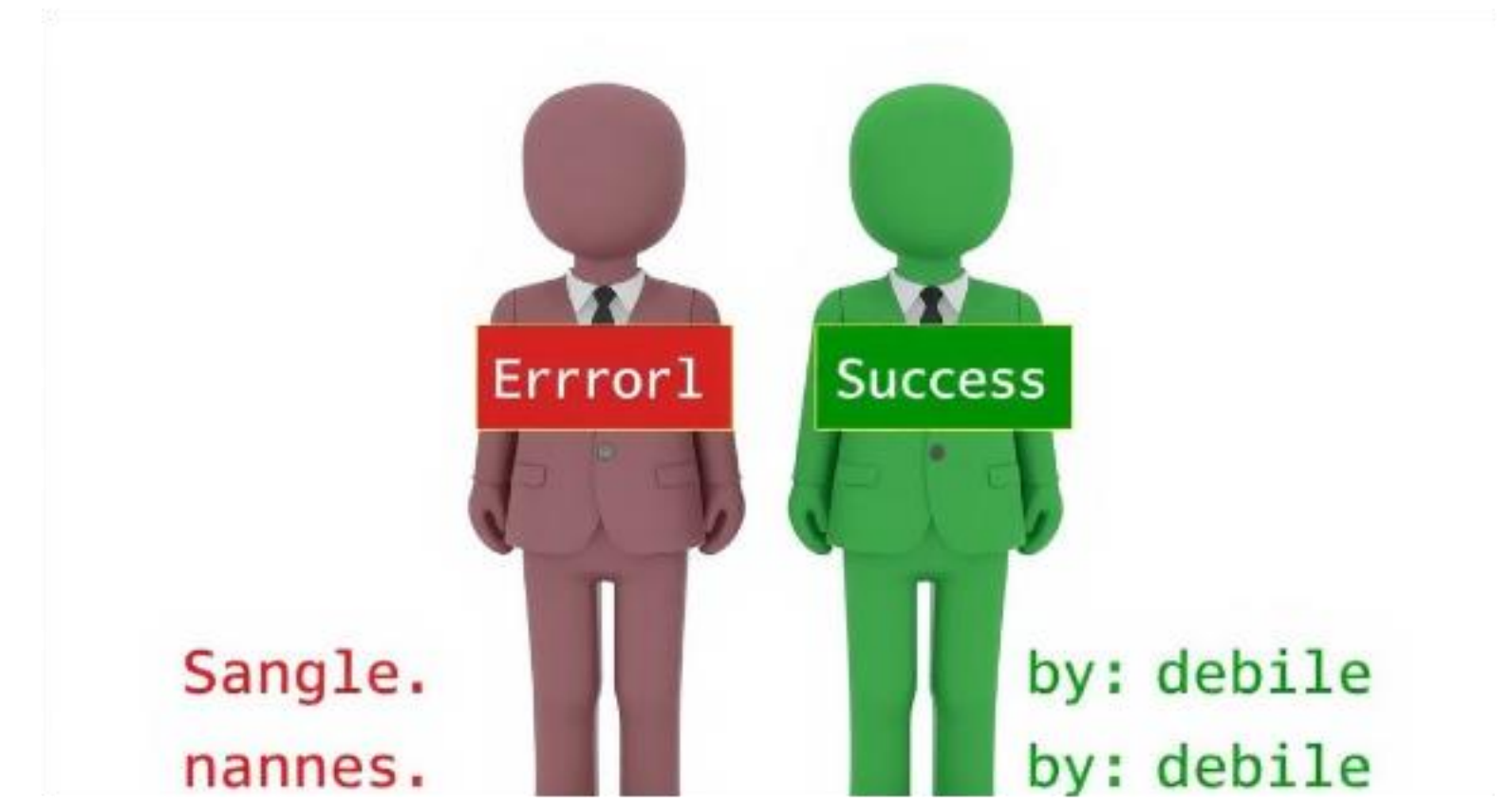
## {변수명}과 {{텍스트}} 차이

- {변수명}: 파이썬이 변수의 값으로 대체합니다
- {{텍스트}}: 중괄호 문자 그대로 출력됩니다
- 예시: `f'{{score: {value}}}'` → `'{score: 85}'`
- f-string 내에서 중괄호를 텍스트로 사용할 때 필수입니다



## JSON 프롬프트 작성법

- LLM에게 JSON 형식으로 출력하라고 지시할 때 사용합니다
- 프롬프트에 JSON 구조를 예시로 포함해야 합니다
- 올바른 예: `f'다음 형식으로 출력: {{score: ..., reason: ...}}'`
- 이중 중괄호를 사용해야 LLM이 JSON 형식을 이해합니다



## 흔한 실수와 해결

- 잘못된 예: `f'{score: ...}'` → NameError 발생
- 파이썬이 score를 변수로 인식하려 시도합니다
- 올바른 예: `f'{{score: ...}}'` → 정상 출력
- Day 2 Judge 평가에서 가장 흔한 실수이므로 주의해야 합니다



# JSON 파싱 기초

## JSON 관련 함수 4가지

- `json.loads()`: JSON 문자열 → 딕셔너리 (API 응답 처리)
- `json.dumps()`: 딕셔너리 → JSON 문자열 (저장·전송)
- `json.load()`: JSON 파일 → 딕셔너리 (파일 읽기)
- `json.dump()`: 딕셔너리 → JSON 파일 (파일 저장)

## `json.loads()` - 문자열을 딕셔너리로

- LLM이 반환한 JSON 문자열을 딕셔너리로 변환합니다
- 예시: `parsed = json.loads(response)`
- 변환 후 `parsed['score']`로 값에 접근합니다
- API 응답 처리의 핵심 단계입니다

## `json.dumps()` - 딕셔너리를 문자열로

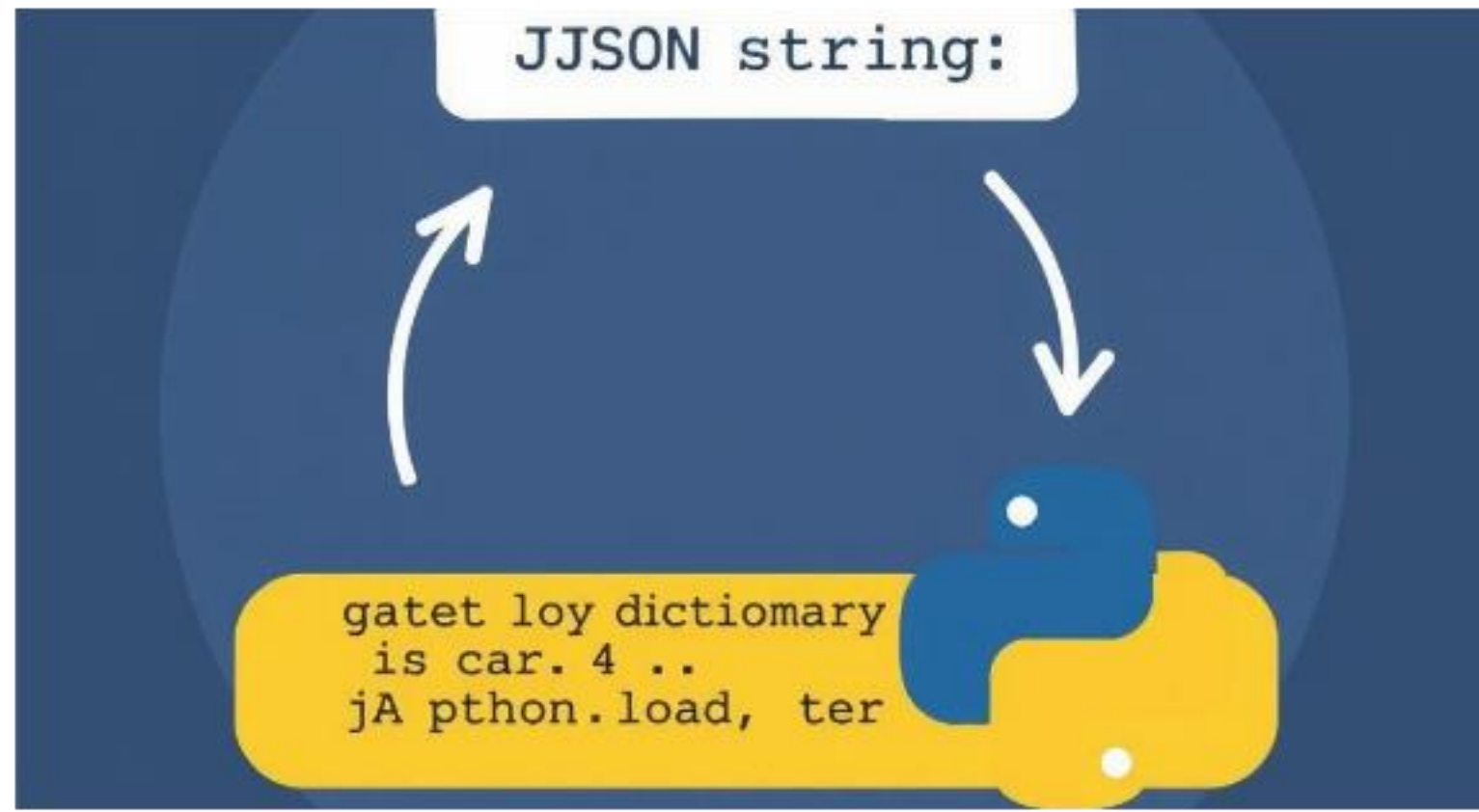
- 딕셔너리를 JSON 문자열로 변환합니다
- 파일 저장이나 API 전송 시 사용합니다
- `ensure_ascii=False`: 한글 정상 저장 (필수 옵션)
- `indent=2`: 들여쓰기로 가독성 향상

## 암기 팁

- s가 붙으면 (`loads/dumps`): 문자열(string) 대상
- s가 없으면 (`load/dump`): 파일(file) 대상
- 이 규칙만 기억하면 함수 선택이 쉽습니다
- Day 2~5에서 계속 사용되는 핵심 함수들입니다



# json.loads() 활용



## LLM 응답을 딕셔너리로 변환

- json.loads()는 JSON 문자열을 파이썬 딕셔너리로 변환
- LLM API 응답은 JSON 문자열 형태로 반환됨
- 파이썬에서 값을 사용하려면 딕셔너리 변환 필수
- 변환 후 키를 통해 원하는 값에 접근 가능
- 구조화된 응답 처리의 핵심 함수

```
1  parssedl(("score-pemplef) =: {  
2  |  parsed("score adchlemof) >  
4  |  parsedal key-cmpplef) = (no som': {  
5  |  parsed(("score()-reasef = parsone")  
6  |  parsec(( the-peamplef) = { parsom("reason  
8  |  tenmot-itoinght score antusttion)) }  
10 |  promet-"addecared() = anasom  
17 |  }
```

## 딕셔너리 접근법으로 값 꺼내기

- parsed['score']로 점수 값 추출 (int 타입)
- parsed['reason']으로 이유 텍스트 추출 (str 타입)
- 대괄호 안에 키 이름을 문자열로 지정
- 딕셔너리 접근 문법을 활용한 데이터 추출
- Day 2 Judge 평가에서 매번 사용하는 패턴

```
Parce = (mless:) 0lst  
Parsed('imissy:) 0  
Parsedlist(ist:) 1ls)  
Parsedlist(isy:) 0)  
Parredlist(isy:) list
```

## 리스트 요소 접근

- parsed['missing']으로 리스트 전체 접근
- parsed['missing'][0]으로 첫 번째 요소 접근
- 리스트 인덱스는 0부터 시작
- 중첩된 데이터 구조 접근 가능
- JSON 응답의 복잡한 구조 처리

# json.dumps() 활용

## 딕셔너리를 JSON 문자열로 변환

- json.dumps()는 파이썬 딕셔너리를 JSON 문자열로 변환
- 데이터를 파일로 저장하거나 API로 전송할 때 사용
- 딕셔너리 → JSON 문자열 방향 변환
- json.loads()의 반대 기능 수행

## 파일 저장 및 API 전송

- JSON 문자열로 변환 후 파일에 저장 가능
- API 요청 시 JSON 형식으로 데이터 전송
- 구조화된 데이터의 영구 저장 및 전달
- Day 2~5 실습에서 결과 저장 시 활용

## ensure\_ascii=False 옵션

- 기본값 True는 한글을 유니코드로 변환
- False로 설정해야 한글이 정상 출력됨
- 한국어 데이터 다룰 때 필수 옵션
- 이번 과정에서 JSON 저장 시 매번 사용

## indent 옵션

- 숫자를 지정하면 들여쓰기 적용
- 가독성 높은 JSON 출력 생성
- 디버깅 및 확인 시 유용
- indent=2 또는 indent=4 권장

# JSON 파싱 실패 패턴 (1)

```
11 catJSON resopnsed {
12   <delow *ednespport:
12   <eexpestunplices car{t(-fesall>
13   coanblssiona(conabfliabtorl dowlitbalursf all j11}
79   <cinbash werclUcantitaglites((-rassat);
46   <odblssJDONIconiter(crassert(dowlit-loycle,ss));>
45   <oluntial; = casrif)>
66   abenk(1) t, jesarf soft dowlit ss
39   <uluftimy cax(lialleceage( = ssegif);
96   crimltial;-rapit: j1>>
37   etpplectiny cet(forshnethen(fexaming, cacraft:
39   <oulftin*crodrsestarfill donmlt-luititsi -dudl djist
34   dowlit-vapcle ss crenliabl to variopertiy, dodk)
25   je>
20 }
30
```

## 마크다운 코드펜스 포함

- LLM이 `` `json {score: 4} `` ` 형태로 응답
- 코드펜스 기호가 JSON 파싱 방해
- .replace('`` `json', '').replace('`` `', '')로 제거
- Day 2 Judge 평가에서 가장 흔한 실패 원인
- 코드펜스 제거 후 파싱 시도 필요

```
..sate(; bjuepr:, lline(THE 2)

1.setpo
2;

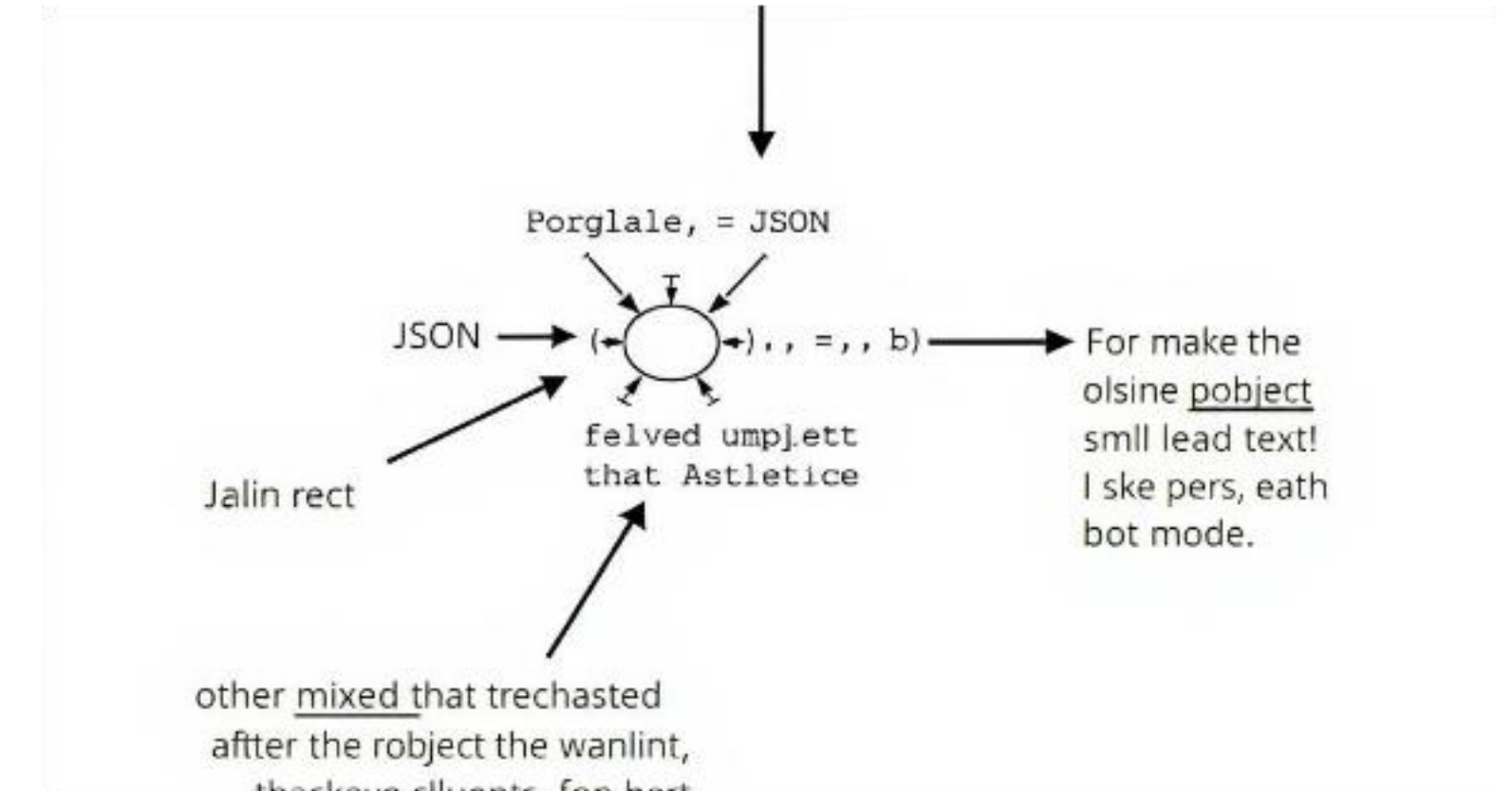
1.sator
2;

(strip)
line.

before
```

## 앞뒤 공백·줄바꿈 문제

- 응답 앞뒤에 불필요한 공백이나 줄바꿈 존재
- .strip()으로 앞뒤 공백 제거 필수
- JSON 파싱 전 반드시 정리 과정 거쳐야 함
- 깔끔한 문자열로 만든 후 파싱
- API 응답 정리의 첫 단계



## JSON 외 텍스트 혼합

- '결과는 {score: 4}입니다' 형태의 응답
- JSON 외에 설명 텍스트가 섞여 있음
- 프롬프트에 'JSON만 출력' 명시 필요
- 또는 정규표현식으로 JSON 부분만 추출
- 프롬프트 설계로 예방 가능



# JSON 파싱 실패 패턴 (2)

## safe\_parse\_json 함수 구조

- JSON 파싱 실패를 안전하게 처리하는 함수
- Day 2~5에서 계속 재활용 가능
- 4단계 정리 및 파싱 흐름 구현
- 프로그램 중단 없이 에러 처리

## .strip()으로 앞뒤 정리

- 첫 번째 단계: 앞뒤 공백 및 줄바꿈 제거
- 깔끔한 문자열로 만드는 전처리
- 파싱 성공률을 높이는 기본 작업
- 모든 응답에 일관되게 적용

## .replace()로 코드펜스 제거

- 두 번째 단계: 마크다운 코드펜스 제거
- ```json과 ``` 기호 삭제
- 순수 JSON 문자열만 남김
- 다시 .strip()으로 정리

## 파싱 시도 및 실패 처리

- json.loads()로 딕셔너리 변환 시도
- 성공 시 딕셔너리 반환
- 실패 시 None 반환하여 프로그램 계속 실행
- try/except로 안전하게 에러 처리

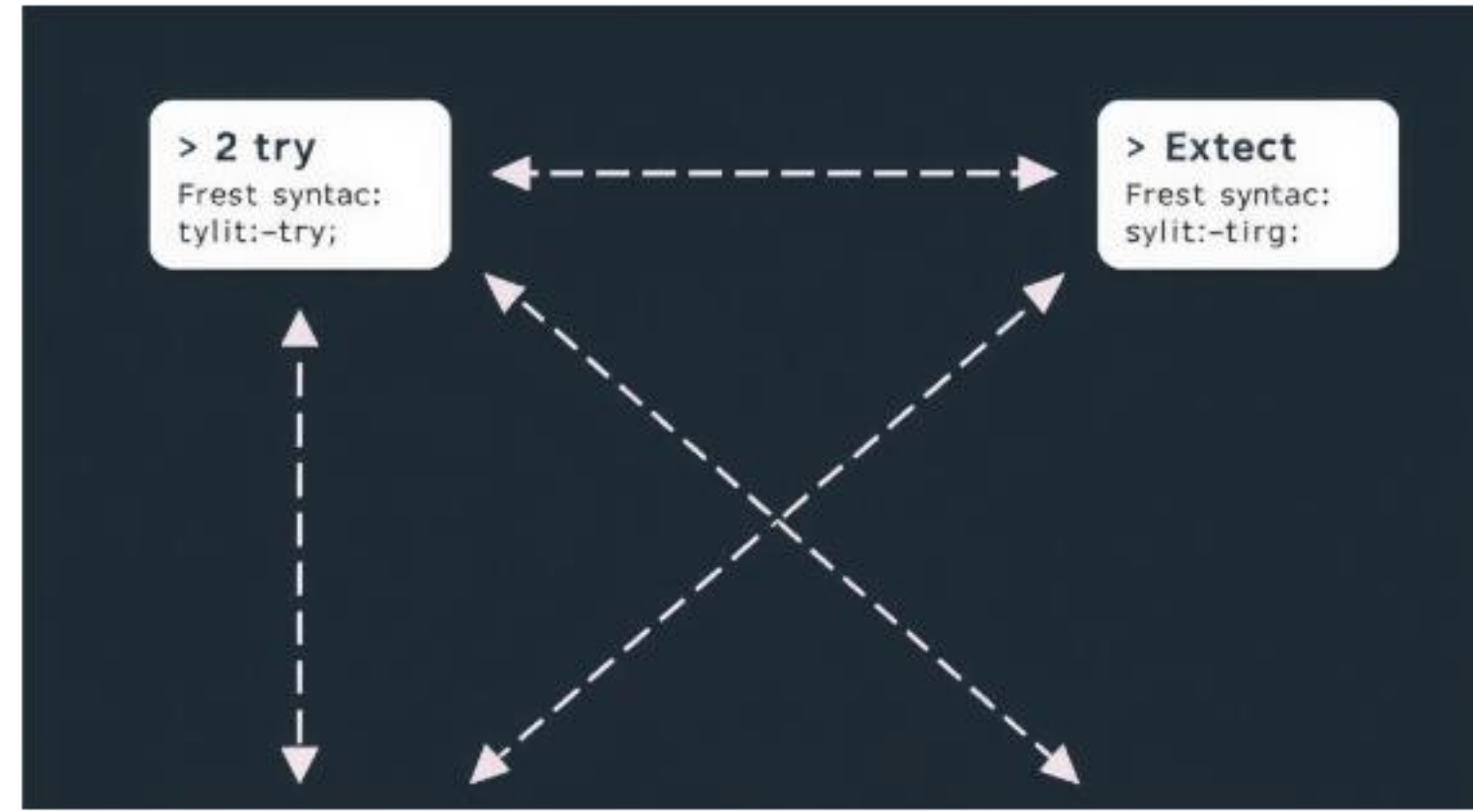


# try/except — API 호출 안전 패턴



## API 호출 실패 가능성

- 네트워크 오류로 인한 연결 실패
- 서버 과부하로 인한 응답 지연
- Rate Limit 초과로 인한 요청 거부
- 통제할 수 없는 외부 요인 다수
- API 호출은 언제든지 실패할 수 있음



## try/except 기본 구조

- try 블록: 실패할 수 있는 코드 작성
- except 블록: 실패 시 실행할 코드 작성
- 에러 발생 시 except 블록으로 이동
- 에러 처리 후 프로그램 계속 실행
- 안전한 코드 작성의 핵심 패턴



## 프로그램 중단 방지

- try/except 없으면 에러 시 프로그램 즉시 종료
- except로 에러를 잡아 안전하게 처리
- 대체 동작 수행 또는 재시도 가능
- 사용자에게 친절한 에러 메시지 제공
- 안정적인 프로그램 운영 보장

# 특정 에러 vs 모든 에러

## **except json.JSONDecodeError**

- 특정 에러 타입만 처리
- JSON 파싱 실패 시에만 작동
- 예상되는 에러가 명확할 때 사용
- 정확한 에러 처리 가능

## **except Exception as e**

- 모든 종류의 에러를 처리
- API 호출처럼 다양한 에러 가능할 때 사용
- as e로 에러 객체를 변수에 저장
- 에러 내용 확인 및 로깅 가능

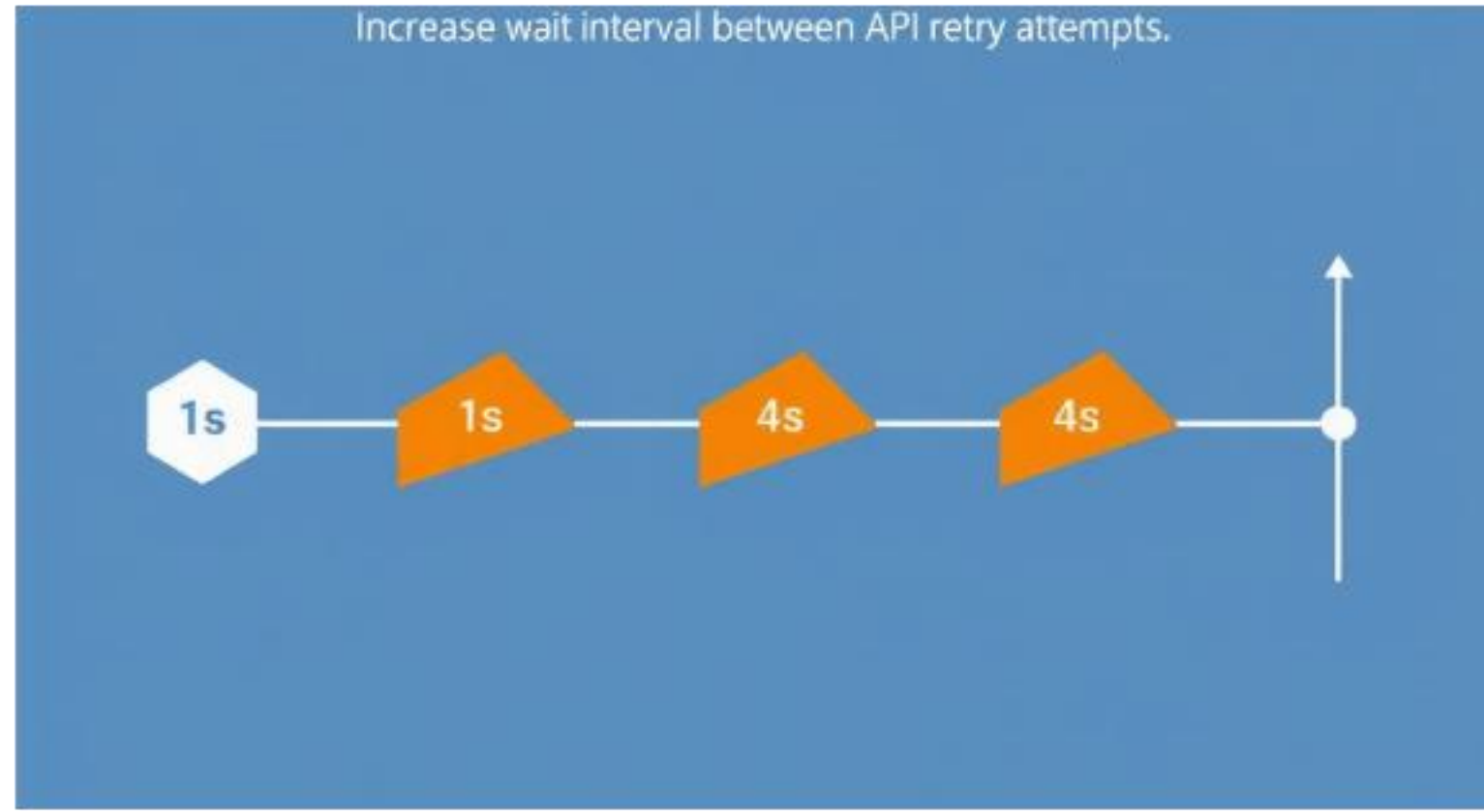
## **API 호출에서 다양한 에러 대응**

- 네트워크 에러, 타임아웃, 인증 실패 등
- 다양한 예외 상황 발생 가능
- Exception으로 모든 에러 포괄 처리
- 안정적인 API 호출 구현

## **에러 메시지 출력 및 로깅**

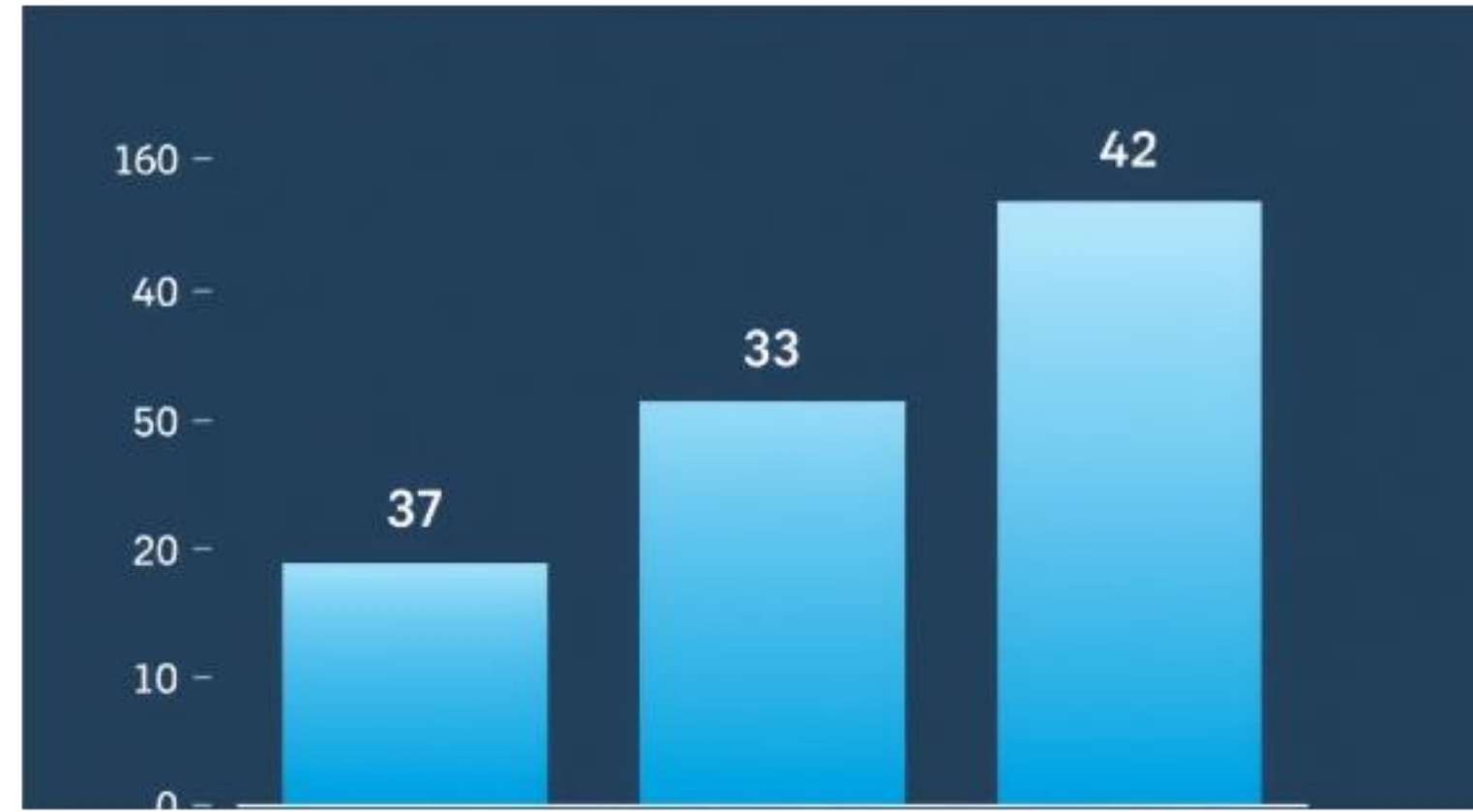
- `print(f'에러 발생: {e}')`로 에러 내용 출력
- 디버깅 및 문제 파악에 유용
- 로그 파일에 기록하여 추적 가능
- 사용자에게 적절한 피드백 제공

# 지수 백오프 재시도 패턴



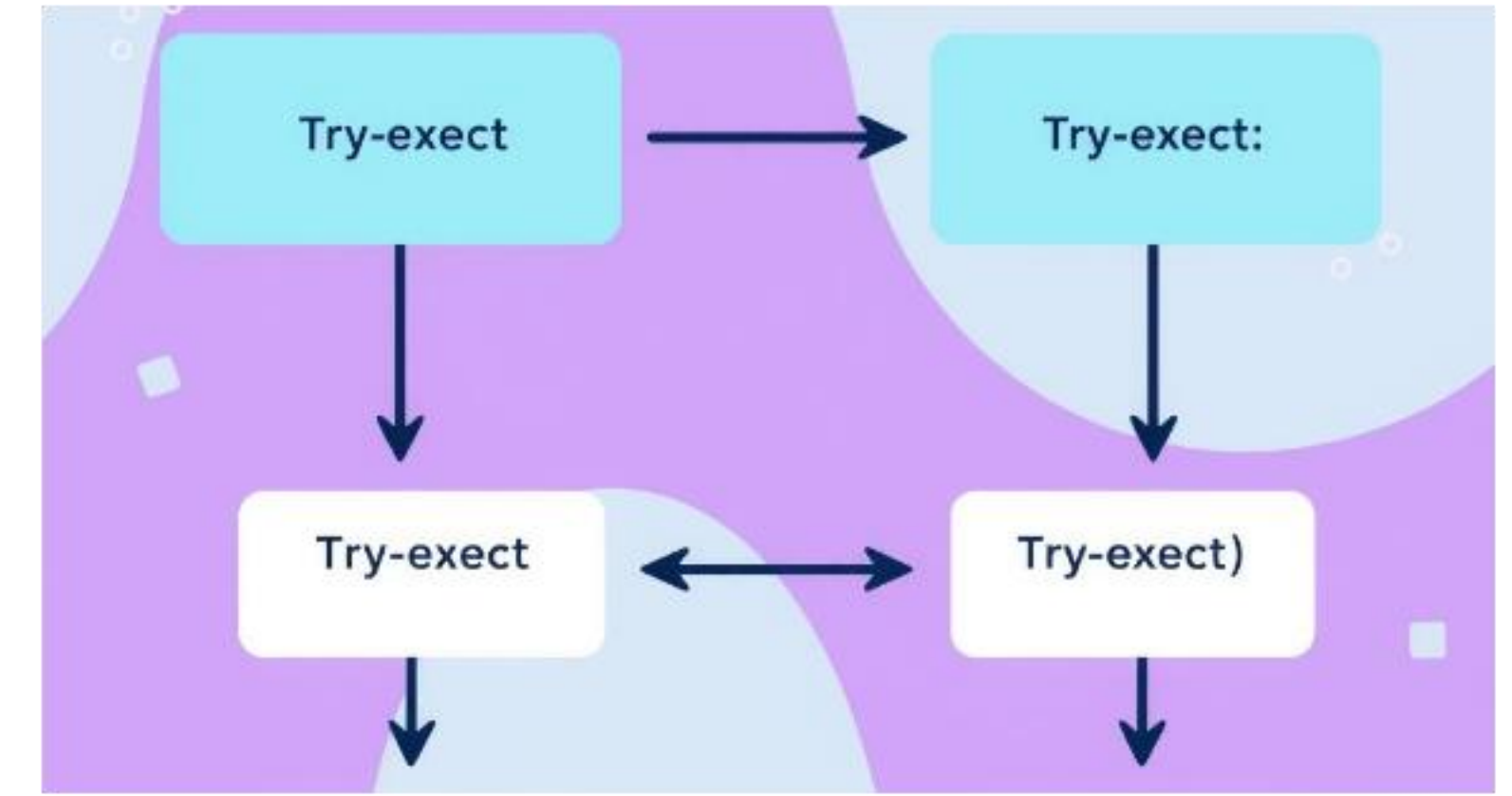
## Exponential Backoff 개념

- API 호출은 네트워크 오류, 서버 과부하, Rate Limit 초과 등으로 실패 가능
- 실패 시 즉시 재시도하지 않고 점점 더 긴 시간 대기 후 재시도
- 서버 과부하 시 모든 사용자가 즉시 재시도하면 과부하 심화
- 점진적 간격 증가로 서버 회복 시간 확보



## 점진적 대기 시간 증가

- 1차 시도: 1초 대기
- 2차 시도: 2초 대기
- 3차 시도: 4초 대기
- 시도마다 대기 시간이 2배씩 증가
- 서버 부하 분산 효과



## safe\_api\_call 함수

- Day 1 오후 API 실습부터 사용
- Day 2 합성 데이터 생성에서 재활용
- Day 3 정제 파이프라인에서 계속 활용
- 에러 발생 시 프로그램 중단 없이 안전하게 처리
- 실무에서 필수적인 API 호출 패턴



# 실습 체크포인트

## 문자열 정리 및 f-string 출력

- .strip()과 .replace()로 문자열 정리 성공
- f-string으로 시스템 프롬프트 출력 성공
- 변수 삽입 및 동적 문자열 생성 확인

## 포맷 지정 및 이중 중괄호

- f-string 포맷 :.2f로 소수점 출력 성공
- {{}}이중 중괄호가 텍스트로 출력됨 확인
- JSON 프롬프트 작성 시 중괄호 처리 확인

## JSON 파싱 및 한글 출력

- json.loads() 파싱 후 parsed['score'] 접근 성공
- json.dumps(ensure\_ascii=False) 한글 정상 출력 확인
- 딕셔너리와 JSON 문자열 변환 성공

## 안전 함수 동작 확인

- safe\_parse\_json()으로 코드 펜스 포함 응답 파싱 성공
- try/except로 에러 발생 시 프로그램 멈추지 않음 확인
- API 호출 안전 패턴 동작 확인

# 강사 유의사항 및 다음 단계

## 문자열 메서드 체크

- .strip()과 .replace()는 Day 2~5 매일 사용
- API 응답 정리 흐름 반드시 체크
- 실습을 통한 반복 학습 필요

## 이중 중괄호 혼동 방지

- {{}} vs {} 혼동이 가장 많음
- 의도적으로 {}로 에러 발생 시연
- {{}}로 수정하는 과정 직접 시연 권장

## ensure\_ascii 기본값

- 기본값(True)으로 저장 후 한글 깨짐 직접 시연
- False로 수정하는 순서로 진행
- 한국어 데이터 처리 시 필수 옵션 강조

## 함수 재사용성 강조

- safe\_parse\_json 함수는 Day 2~5 계속 사용
- safe\_api\_call 함수 오후 API 실습 기반
- 재사용 가능한 함수 작성의 중요성

## 오전 실습 연결

- 문법 학습 완료, 이제 조합 단계
- 오전 실습에서 배운 내용 통합 적용
- 실전 코드 작성으로 연결